

Computation-Adaptive Video Streaming

Zichen Zhu[†], Stefano Petrangeli^{*}, Yao Liu[†], and Sheng Wei[†]

[†]Rutgers University ^{*}Adobe Research

{zichen.zhu, yao.liu, sheng.wei}@rutgers.edu; petrangle@adobe.com

Abstract—Video streaming has become a prevalent multimedia application, which presents significant challenges in delivering high-quality video content to the end users under varying network conditions. The traditional Internet video streaming solution is bandwidth-adaptive, where multiple versions of the video content with distinct qualities are deployed on the streaming servers, and the clients execute adaptive bitrate (ABR) algorithms to determine and request the most suitable version based on the available bandwidth. While bandwidth-adaptive streaming has been adopted for decades, it is still subject to practical limitations due to tremendous storage overhead and inaccurate bitrate adaptations. We propose a novel computation-adaptive video streaming mechanism, namely CAVS, which transitions bandwidth-adaptive streaming to computation-adaptive by leveraging the recent advancements in generative AI technologies (e.g., super-resolution). CAVS stores and streams only the low-resolution video and adopts a computation-adaptive, saliency-based partial super-resolution approach on the client to maximize the perceptual video quality. Furthermore, we employ a Proportional-Derivative (PD) controller to adapt the super-resolution workload per frame, ensuring real-time playback under varying computation capacities. We evaluate CAVS using 16 videos with varying GPU capacities on the client, which shows computation-adaptive viewing quality, real-time performance, and significantly reduced bandwidth and storage usage.

I. INTRODUCTION

Video streaming has been a prevalent multimedia application across platforms such as YouTube, Twitch, and Netflix, with rapid advancements toward high resolution (e.g., 4K and 8K) and immersive experience (e.g., 360-degree video). While enhancing user experience, these developments have significantly increased network bandwidth demands. To address the bandwidth challenge, the existing streaming systems commonly employ bandwidth-adaptive streaming, adopting adaptive bitrate (ABR) algorithms to adjust video quality based on varying network conditions [1]–[3]. However, ABR algorithms are not always reliable: bandwidth estimations can be inaccurate, multi-client competition introduces unfairness, and rapid network fluctuations may outpace algorithm responsiveness. Consequently, users may still experience re-buffering, quality drops, or frame stalls despite the ABR algorithms. Furthermore, with multiple versions of the same video, bandwidth-adaptive streaming incurs significant storage overhead at the server.

To mitigate the challenges in bandwidth-adaptive streaming, we propose CAVS, a computation-adaptive video streaming mechanism. In lieu of relying on real-time bitrate adaptation, CAVS always streams a low-resolution version of the video to minimize the storage and bandwidth usage. On the client side, CAVS selectively enhances the tiles of each frame using neural

network-based super-resolution based on the tiles’ saliency weights, adapting in real time to the client’s current computation capacity. By combining low-resolution transmission with tile-based super-resolution, CAVS effectively decouples bandwidth consumption from output video quality.

We implement CAVS as a full-stack video streaming system featuring three key components: 1) tile-based super-resolution for fine-grained enhancement; 2) saliency-based tile prioritization to focus computation on perceptually important regions; and 3) a Proportional-Derivative (PD) controller to adapt the SR workload per frame, ensuring smooth real-time playback under varying computation capacities. To evaluate CAVS, we conduct extensive experiments under both static and dynamic GPU capacity scenarios. The evaluation results show that CAVS achieves up to 89% bandwidth reduction compared to the baseline full-resolution streams while maintaining computation-adaptive perceptual quality and real-time streaming performance. In summary, this paper makes the following contributions:

- We propose CAVS, a novel computation-adaptive streaming mechanism, which addresses the challenges in traditional bandwidth-adaptive streaming by transmitting low-resolution video and adaptively recovering the quality at the client side.
- We design a tile-based saliency-aware super-resolution pipeline controlled by a lightweight PD controller for real-time responsiveness to varying computation capacities.
- We implement and evaluate CAVS in a video streaming system with varying client computation capacities.

II. BACKGROUND AND RELATED WORK

A. Bandwidth-adaptive video streaming

The development of bandwidth-adaptive streaming using ABR algorithms has progressed significantly with the advancement and widespread deployment of HTTP adaptive streaming (HAS), adopting buffer-based [1], control-based [2], and deep learning-based [3] approaches to dynamically adjust the bitrate of the video stream based on the varying network bandwidth. These diverse ABR algorithms are widely supported and utilized in video streaming standards, such as DASH [4] and HLS [5]. Despite their advantages, ABR algorithms face several challenges. First, they lack a reliable bandwidth estimation method, as available bandwidth is influenced not only by local tasks but also by other devices in the local network and throughout the entire path to the server. Second, algorithms may overestimate available bandwidth in multi-client scenarios

when they lack awareness of neighboring devices. Third, they often fail to respond quickly to rapid network changes. These factors can render ABR algorithms unreliable [6].

B. Super-resolution

Super-resolution (SR) has undergone rapid development as an image upscaling method, evolving beyond traditional interpolation techniques with the advent of deep learning. Dong *et al.* introduced SRCNN [7], which applies convolutional neural networks (CNNs) to image super-resolution. Shi *et al.* introduced sub-pixel convolution and pixel shuffle in the ESPCN model [8], a CNN-based architecture that significantly reduced computational demands and improved processing speed, thereby enabling SR on devices with limited computational resources. Wu *et al.* introduced ASRSR [9], which applies different SR configurations to video chunks based on bandwidth and predicted quality of experience (QoE). In contrast to ASRSR's approach of applying configurations to entire frames or groups of frames, Li *et al.* proposed EVASR [10], which divides frames into tiles and selectively applies SR to certain tiles on the edge server based on saliency weights, user counts, and current edge server configurations.

III. SYSTEM DESIGN

A. System Overview

We develop a computation-adaptive video streaming system, namely CAVS, to address the challenges in traditional bandwidth-adaptive streaming. CAVS integrates three essential components: tile-based partial super-resolution, offline saliency weight calculation for intra-frame tile prioritization, and frame duration control. It ensures rapid adaptation to changes in client-side computation resources, such as the available GPU capacity, allowing for swift recovery to target frame durations. Fig. 1 illustrates the overall workflow of CAVS, which comprises a *Server* and a *Player*. The *Server* is responsible for transcoding the input video into lower resolution to reduce bandwidth usage between the *Server* and *Player*, as well as calculating the saliency weights to guide tile-based super-resolution on the *Player*. The *Player* integrates tile-based super-resolution and employs a frame duration control component to ensure real-time playback.

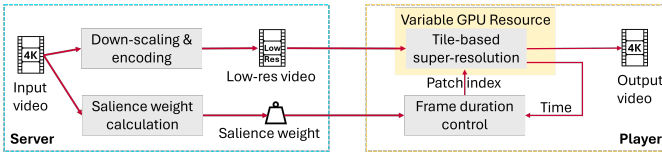


Fig. 1: Workflow of the proposed CAVS system.

As shown in Fig. 1, the general workflow of the CAVS solution is as follows. When the high-resolution video is submitted to the *Server*, it first calculates the per-frame saliency weight. Then, it divides each frames according to a predefined tile layout and further calculates the saliency weight of each tile. The tile-based saliency weights are then packed as an additional file with the video file. Then, the *Server*

further transcodes the input video into lower resolution and deploys the video-weight pair on the *Server* for the *Player* to stream. The *Player* downloads the weight file and starts streaming the low resolution video in the video-weight pair. When a frame is received at the *Player* from the stream, it is first divided into tiles using the same layout as on the *Server*. These tiles are then sorted based on the corresponding weights in the additional file. Based on the output of the frame duration control, the top u tiles are sent to the tile-based super-resolution process, and the rest are upscaled to the same size as the super-resolution using bicubic interpolation. After all tiles are upscaled, the *Player* re-assembles them into an upscaled frame for display, as well as record the current timestamp for the frame duration control unit to adjust the number of tiles with super-resolution in the next frame.

B. Frame Duration Control

We assume that the computational requirements of super-resolution are linearly dependent on the number of input pixels. This implies that, for an equal number of pixels, the processing time under the same computation capacity remains relatively consistent. Furthermore, for tiles of identical size, an increase in the number of tiles would lead to a proportional increase in the processing time. Additionally, considering that in CAVS each super-resolution computation is only related to the current input pixels and independent of past or future ones, we can conclude that the super-resolution computation is time-invariant under a fixed computation capacity. Therefore, when the input of tile-based super-resolution is the number of tiles and the output is the computation time, we can approximately model this system as a linear time-invariant (LTI) system.

Given that the tile-based super-resolution exhibits the LTI property, employing Proportional-Integral-Derivative (PID) control [11] to ensure stable tracking of the desired output value is a common control strategy. Unlike typical PID control, the disturbances in this system originate from external resource constraints (e.g., increases or decreases in available GPU resources) rather than changes in the desired output value. Consequently, one of the PID inputs, $r[n]$, is a constant function. Considering that potential steady-state errors in the system may arise from excessive or insufficient computational resources, we employ only Proportional-Derivative (PD) control, excluding integral control, as shown in Fig. 2. This is because, in cases of resource surplus, super-resolution has already exceeded the desired task and thus requires no additional adjustments. In resource-deficient scenarios, the system degrades to interpolation upscaling, at which point we cannot further reduce the number of tiles. Considering the complexity of discrete implementation, we simplify the PD control in Fig. 2 in the following form:

$$u[n] = u[n-1] + K_p \epsilon[n] - K_d (\epsilon[n] - \epsilon[n-1]) \quad (1)$$

where $u[n]$ is the number of tiles with super-resolution in

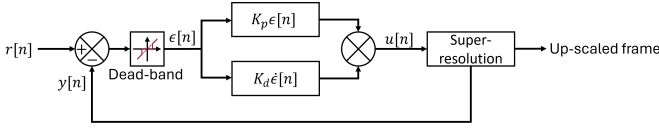


Fig. 2: Block diagram of the *Frame Duration Control*.

frame n , K_p and K_d are factors, and $\epsilon[n]$ is the error between the target and the measurement with a dead-band:

$$\epsilon[n] = \begin{cases} 0 & \text{if } D_{low} < r[n] - y[n] < D_{high} \\ r[n] - y[n] & \text{else} \end{cases} \quad (2)$$

where $r[n]$ is the expected frame duration at frame n , $y[n]$ is the most recent actual frame duration when frame n is being processed, and D_{low} and D_{high} are the lower and upper boundaries of the dead-band, respectively.

In this controller, the adjustable parameters are K_p , K_d , D_{high} , and D_{low} . K_p represents the controller's response intensity to the current error, and K_d indicates the control over the rate at which the error increases or decreases. We desire a relatively large K_p to enable quick responses, thereby reducing the number of frames exceeding the desired frame duration when external resource constraints change. In the meantime, we set an appropriate K_d to mitigate overshooting caused by rapid responses. Additionally, we aim for a suitable dead-band to prevent fluctuations in $u[n]$ when $y[n]$ approaches but remains slightly below $r[n]$, thus avoiding repeated oscillations.

IV. SYSTEM IMPLEMENTATION

Fig. 3 illustrates the implementation of our proposed CAVS system, detailing the technical components within both the *Server* and the *Player*.

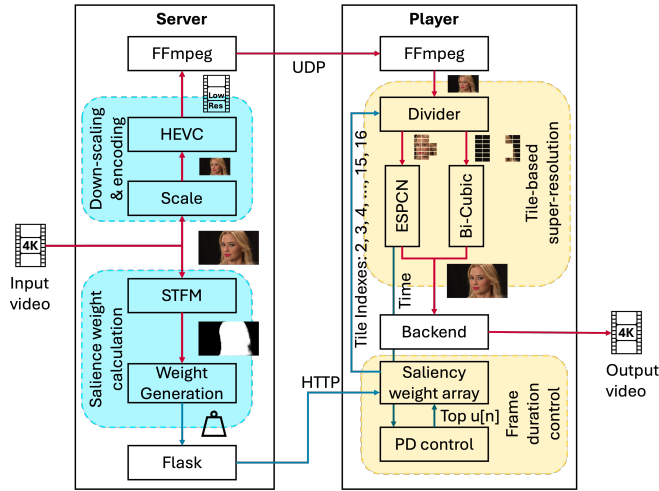


Fig. 3: System implementation of CAVS.

A. Server

After the input video is loaded onto the *Server*, two concurrent processes are initiated: *saliency weight calculation* and *down-scaling & encoding*. Following the completion of both

processes, the input video and all intermediate files (e.g., the saliency maps) are deleted, retaining only the low-resolution video and the saliency weights on the *Server*.

When a video-weight pair is prepared, the *Server* makes it available for the *Player* to retrieve. The weight file is served via HTTP using the *Flask* [12] framework. Upon each weight file request, the corresponding video in the video-weight pair is loaded and pinned in memory to facilitate streaming and minimize I/O wait time, which could otherwise affect execution time and frame duration control. Once the video is loaded into memory, an *FFmpeg* [13] process is initiated to stream the corresponding video in the video-weight pair using the UDP protocol.

1) *Saliency Weight Calculation*: The saliency weight calculation process generates saliency weights for each tile in the video. Initially, the video is converted into individual frames in PNG format. An STFM model [14] then produces a grayscale saliency map for each frame. Subsequently, these saliency maps are processed using EVASR [10] to derive tile-based saliency weights, adhering to a 6x6 tile layout that is also implemented in the *Player*. Finally, the tile-based weights are saved as a file, and all intermediate images are deleted.

2) *Down-Scaling & Encoding*: The down-scaling & encoding process is responsible for reducing the resolution and transcoding the input video to a low-resolution format. *FFmpeg* is utilized for both down-scaling and encoding tasks. Initially, the input video is reduced by a factor of 4 (e.g., from 3840x2160 to 960x540) using the built-in video filter *scale*. It is then encoded with HEVC [15] for streaming to the *Player*.

B. Player

The *Player* is implemented in Python, primarily using the *PyTorch* library for GPU-based computation and the *subprocess* library to run *FFmpeg* in the background for I/O operations. Initially, the *Player* initiates a receiving *FFmpeg* instance with GPU-based *hevc_cuvid* codec before fetching the saliency weights from the *Server*. This sequence prevents the *Server*-side *FFmpeg* from starting prematurely and potentially missing initial frames. The *Player* then retrieves saliency weights for tile-based super-resolution and triggers the *Server*-side *FFmpeg* to begin video streaming. These weights are subsequently loaded into the *Frame Duration Control* component.

Upon receiving frames from the *Server*, the *FFmpeg* instance outputs them to the standard output pipe in the YUV444P pixel format. The *Player* reads this output and converts the raw data into a tensor for subsequent operations. This tensor is then processed by the *Tile-Based Super-Resolution* component for upscaling, with the *Frame Duration Control* component determining the number and selection of tiles for super-resolution. Finally, the upscaled tensor is converted back to the image format and sent to various backends for display and quality evaluation.

1) *Tile-Based Super-Resolution*: The tensor is initially processed by a divider, which segments the frame into tiles based on the 6 x 6 layout previously described in Section IV-A.

The divider also receives a list of tile indices from the *Frame Duration Control*, indicating which tiles should undergo super-resolution. The designated tiles are then processed by the ESPCN model for super-resolution, following the tile-based approach outlined by EVASR [10], while the remaining tiles undergo bicubic interpolation using the torchvision library. Since the ESPCN model operates solely on the luminance channel, the entire frame tensor is first upsampled using bicubic interpolation and, subsequently, the luminance channel of the upsampled frame is overwritten by the ESPCN results. The tensor is then converted back to the raw data format for transmission to the backend. Immediately before this transmission, the *Player* records the current timestamp and updates the *Frame Duration Control*. Throughout this process, all tensor operations are performed on the GPU until the final conversion to raw data.

2) *Frame Duration Control*: The *Frame Duration Control* component ensures real-time processing of the *Tile-Based Super-Resolution* component by regulating the number of tiles processed and maintaining quality by selecting tiles based on saliency weights. As shown in **Algorithm 1**, when a timestamp is updated, the controller calculates the execution time (**diff**) of the current frame and the **error** between the target frame duration ($r[n] = 33$) and **diff**. This error is then constrained by the dead-band boundaries (D_{low}, D_{high}) before derivative calculation. Subsequently, an adjustment (**adjust**) is computed and applied to the previous tile count, indicating the number of tiles to be super-resolved. The resulting tile count is further constrained to the range $[1, 36]$ before being sent to the *Tile-Based Super-Resolution*. Finally, the *Frame Duration Control* sorts the saliency weights of the corresponding tiles, selects the top tile indices based on the calculated tile count (**tile**), and returns this selection to the *Tile-Based Super-Resolution*.

Algorithm 1 Implementation of PD control in the *Frame Duration Control* component.

```

1: Input: Timestamp of displaying the current frame, ts
2: Output: Number of tiles to be super-resolved, tile
3: diff  $\leftarrow$  ts - last_ts
4: last_ts  $\leftarrow$  ts
5: error  $\leftarrow$   $r[n] - \text{diff}$   $\triangleright r[n] = 33$ 
6: if  $D_{low} < \text{error} < D_{high}$  then
7:   error  $\leftarrow$  0  $\triangleright$  Dead-band
8: end if
9: derivative  $\leftarrow$  error - last_error
10: adjust  $\leftarrow$   $K_p * \text{error} - K_d * \text{derivative}$ 
11: last_error  $\leftarrow$  error
12: tile  $\leftarrow$   $\max(1, \min(36, \text{tile} + \text{adjust}))$ 

```

V. EXPERIMENTAL RESULTS

A. Experimental Setup

We deploy CAVS in a video streaming system, where the *Server* and the *Player* are connected via a wired local network. The *Server* is equipped with an 8-core vCPU and 16 GiB

of memory, and the *Player* has a 32-core vCPU, 64 GiB of memory, and an NVIDIA A6000 GPU to support decoding and super-resolution. The STFM model used in Section IV-A utilizes pre-trained weights published in [16]. The ESPCN model employed in Section IV-B is trained on the VOC2012 dataset [17] with a scale factor of 4.

Test Videos. We utilize all 16 videos from the UVG dataset [18] as inputs. We maintain the original resolution (3840x2160) for our evaluation but disregard the original frame rate, processing frames sequentially to measure the actual execution time of each frame. Only the 8-bit version of the videos is used, as they are more common and better suited to our neural network configurations. Prior to transmission to the *Player*, the input videos from the dataset are encoded from raw YUV format using HEVC with the medium preset and constant quality mode ($qp = 28$). The backend in the *Player* preserves the output frames in the same resolution, bit-depth, and pixel format (YUV420P) as the input video in the dataset for consistent evaluation.

Parameter Tuning. To evaluate CAVS performance under varying GPU resource availability, we define 3 GPU capacity levels, limiting the maximum GPU clock frequency using nvidia-smi [19]: **GPU-Low** (400 MHz), **GPU-Mid** (800 MHz), and **GPU-High** (Unlimited). As mentioned in Section III-B, 4 parameters are associated with *Frame Duration Control*: D_{low} , D_{high} , K_p , and K_d . In the subsequent evaluations, we use the following configurations: $D_{low} = 0$, $D_{high} = 3$, $K_p = 0.5$, and $K_d = 0.25$. These parameters are primarily determined through manual tuning. The dead-band parameters aim to treat all frames with durations in the range of (30, 33) ms as requiring no adjustment, thus avoiding duration fluctuations around 33 ms. This configuration introduces a 2-3 tile margin under **GPU-Mid** and **GPU-High**, where one tile takes on average 1.4 ms and 1.0 ms, respectively. This approach trades off the quality benefit from the last 2-3 tiles for stable quality while allowing the controller to maximize quality under the **GPU-Low** cap, where one tile takes an average of 8.7 ms. The PD controller's parameters are determined through a two-step process: 1) tuning K_p to stabilize the execution time at the target $r[n] = 33$ with initial overshoot, and 2) adjusting K_d to smooth the initial overshoot, gradually approaching $r[n] = 33$.

Test Cases. We conduct experiments in two categories: *Constant* and *Dynamic*. In the *Constant* category, the GPU capacity remains at a fixed level. In the *Dynamic* category, the GPU capacity changes according to specific patterns.

- **C1: Constant-Low**: GPU capacity remains at **GPU-Low** throughout the entire video.
- **C2: Constant-Mid**: GPU capacity remains at **GPU-Mid** throughout the entire video.
- **C3: Constant-High**: GPU capacity remains at **GPU-High** throughout the entire video.
- **D1: Drop**: GPU capacity decreases from **GPU-High** to **GPU-Low** at the 12th second.
- **D2: Drop-Recover**: Similar to *Drop*, but the GPU capacity returns to **GPU-High** 5 seconds after the initial decrease.

- **D3: Low-Mid-High:** GPU capacity decreases from **GPU-High** to **GPU-Low** at the 12th second, increases to **GPU-Mid** at the 17th second, and finally returns to **GPU-High** at the 22nd second.
- **D4: Random:** GPU capacity alternates among the 3 levels every second, maintaining the same pattern across all videos.

B. Quality Evaluation

We employ VMAF [20] (version: 4db7c0c8, model: 4k_v0.6.1) to assess the super-resolution quality of the proposed system. The evaluation compares the input video on *Server* before down-scaling (reference) with the output video on *Player* after super-resolution (measurand). Table I presents the average VMAF scores over all frames for the 7 cases across 16 videos. In the *Constant* category, VMAF scores increase with higher GPU capacities as anticipated. The average improvement from *Constant-Low* to *Constant-High* is 25.4, with minimum and maximum increments of 2.2 (HoneyBee) and 45.5 (FlowerKids), respectively. The average VMAF scores for *Constant* cases are 43.9 (*Constant-Low*), 60.9 (*Constant-Mid*), and 69.3 (*Constant-High*).

TABLE I: Average VMAF scores.

Video	Constant			Dynamic			
	C1	C2	C3	D1	D2	D3	D4
Beauty	43.4	50.5	59.0	50.1	55.3	53.8	50.2
Bosphorus	48.6	65.5	69.0	59.1	63.6	63.7	59.2
CityAlley	56.8	69.6	73.2	64.3	67.8	67.7	63.3
FlowerFocus	41.6	56.3	73.5	58.2	65.4	63.6	56.3
FlowerKids	23.4	51.2	68.9	47.1	58.5	57.5	44.4
FlowerPan	45.2	57.8	68.7	56.1	63.1	60.0	53.8
HoneyBee	70.1	71.9	72.4	71.0	71.7	71.6	70.9
Jockey	38.7	57.3	76.1	48.9	65.8	62.4	55.4
Lips	54.6	71.8	72.8	62.8	68.0	68.0	65.6
RaceNight	33.9	70.0	76.8	57.4	66.3	67.4	57.7
ReadySteadyGo	39.7	64.4	80.8	53.5	70.6	70.2	60.3
RiverBank	35.8	50.7	54.1	45.6	49.8	48.9	45.3
ShakeNDry	56.6	60.9	61.5	61.3	61.4	61.3	59.5
SunBath	25.5	54.4	69.4	64.2	65.0	65.3	48.4
Twilight	54.8	64.3	71.3	60.5	66.3	65.6	61.6
YachtRide	32.9	56.9	62.0	46.7	54.3	54.8	47.3
Average	43.9	60.9	69.3	56.7	63.3	62.6	56.2

In the 4 *Dynamic* cases, the VMAF scores fall between *Constant-Low* and *Constant-High*, as expected. The *Drop* case averages 56.7. *Drop-Recover* averages 63.3, higher than *Drop* due to its longer duration at **GPU-High**. *Low-Mid-High* averages 62.6, slightly below *Drop-Recover*, as it operates at **GPU-Mid** instead of **GPU-High** when recovering from **GPU-Low**. The *Random* case averages 56.2, marginally lower than *Constant-Mid*, as the recovery from lower capacities introduces temporary score reductions.

C. Execution Time Evaluation

We measure the execution time for each frame to evaluate the real-time performance of CAVS, as shown in Table II. The execution times are calculated by subtracting the previous frame's timestamp from the current frame's output timestamp.

Notably, the first frame's execution time is consistently set to 33 ms, as any initial delay would not be perceptible to the user. Generally, all cases maintain the execution times close to the 33-ms target.

TABLE II: Average per-frame execution time (ms).

Video	Constant			Dynamic			
	C1	C2	C3	D1	D2	D3	D4
Beauty	33.9	33.7	32.7	34.0	33.9	33.6	34.1
Bosphorus	34.0	33.8	32.5	34.3	33.6	33.8	34.4
CityAlley	33.9	33.7	32.5	33.9	33.9	33.4	34.5
FlowerFocus	33.9	33.7	32.1	34.2	33.8	33.7	34.0
FlowerKids	33.9	33.6	32.0	34.4	34.1	33.7	34.2
FlowerPan	34.1	33.9	32.6	33.7	33.5	33.5	33.9
HoneyBee	33.7	33.7	32.0	34.1	33.8	34.1	34.1
Jockey	34.3	33.8	33.0	34.3	34.0	33.6	34.1
Lips	34.1	33.7	32.4	34.2	33.5	33.7	34.3
RaceNight	33.9	33.8	32.5	34.2	33.8	33.8	34.3
ReadySteadyGo	34.0	33.7	32.3	34.3	33.6	33.7	34.0
RiverBank	34.3	33.7	32.3	33.7	33.6	33.5	34.1
ShakeNDry	34.3	33.7	33.2	34.3	34.4	34.5	35.4
SunBath	34.6	33.8	32.8	34.7	34.1	34.1	35.0
Twilight	34.0	33.8	32.7	34.2	33.6	33.9	34.4
YachtRide	34.0	33.8	32.6	33.8	33.7	33.7	34.4
Average	34.1	33.7	32.5	34.1	33.8	33.8	34.3

In the *Constant* category, the average execution times are 34.1 ms (*Constant-Low*), 33.7 ms (*Constant-Mid*), and 32.5 ms (*Constant-High*). The slightly higher averages for *Constant-Low* and *Constant-Mid* are attributed to the dead-band setting discussed in Section III-B. When a frame falls in the dead-band, the error is set to zero, nullifying the P-term ($K_p \epsilon[n]$). However, the D-term ($K_d \dot{\epsilon}[n]$) retains a residual that can drive the controller to further increase the tile count, overshooting the target $r[n] = 33$. This effect is more pronounced with lower GPU capacities, as longer per-tile execution times (8.7 ms, 1.4 ms, and 1.0 ms for *Constant-Low*, *Constant-Mid*, and *Constant-High*, respectively) lead to narrower dead-band margins, making overshooting more likely.

In the *Dynamic* category, the execution times for *Drop*, *Drop-Recover*, *Low-Mid-High*, and *Random* are 34.1 ms, 33.8 ms, 33.8 ms, and 34.3 ms, respectively. Beyond the aforementioned reasons, the GPU capacity changes contribute to execution time overruns. When the GPU capacity drops from a higher level (e.g., **GPU-High**), the controller can only respond after detecting an overtime execution in the current frame. The period between the overtime detection and the controller's gradual tile reduction introduces additional overhead in the *Dynamic* category. Consequently, the *Random* case performs the worst due to frequent GPU capacity fluctuations, and *Drop* is the second-worst, operating predominantly under **GPU-Low**, similar to *Constant-Low*.

D. Bandwidth Usage Evaluation

To evaluate the bandwidth savings achieved by CAVS, we measure the bitrate of the low-resolution video streamed by CAVS and compare it with the bitrate of the original video streaming (i.e., the baseline). For a fair comparison, in the

baseline case, the video is encoded using the same codec and parameters as CAVS (described in Section V-A) but at its original resolution. In both cases, the bitrate is measured at 30 FPS instead of the original frame rate. Fig. 4 shows the bandwidth results comparing CAVS with the baseline.

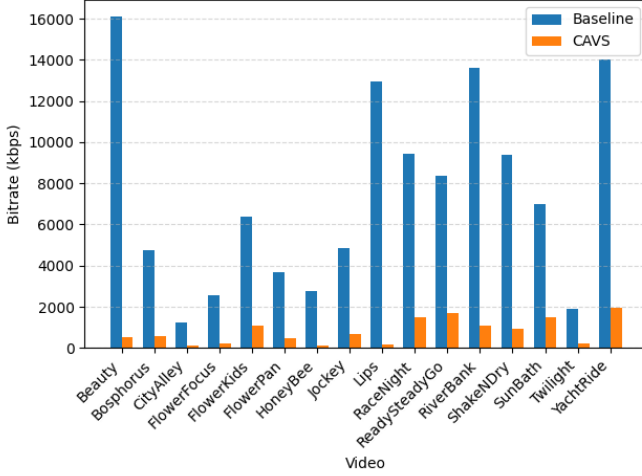


Fig. 4: Bitrate comparison between CAVS and the baseline.

The average bandwidth reduction from the baseline across all videos is 6,640.9 kbps (89%). The maximum reduction is observed in the “Lips” video, decreasing from 12,930.3 to 148.0 kbps (99%). This video features a close-up makeup application with a simple background and minimal, slow motion. The VMAF scores for this video in the 3 GPU capacities (**GPU-Low**, **GPU-Mid**, and **GPU-High**) are 54.6, 71.8, and 72.8, respectively, with tile counts increasing from 3.6 to 23.7 and 33.8. The smallest reduction is seen in the “SunBath” video, decreasing from 7,017.7 to 1,518.9 kbps (78%). This video depicts slowly swaying tree branches. Its VMAF scores for the 3 GPU capacities are 25.5, 54.4, and 69.4, with tile counts of 2.8, 21.7, and 32.6, respectively.

In the *Baseline* category, the highest bitrate video is “Beauty”, featuring a close-up of a female face against a simple background with slow motion. Its bitrate is 16,115.8 kbps, which is reduced to 512.4 kbps by CAVS (97% savings). The VMAF scores of CAVS for this video across different capacities are 43.4, 50.5, and 59.0, with tile counts of 5.7, 25.6, and 33.6, respectively. In the *CAVS* category, the highest bitrate video is “YachtRide,” showing a close-up of a cruising boat. Its bitrate is reduced from 13,999.2 to 1,959.9 kbps by CAVS (86% savings). The VMAF scores of CAVS for this video are 32.9, 56.9, and 62.0, with tile counts of 3.8, 24.9, and 33.6 across the 3 GPU capacities. The lowest bitrate in both *Baseline* and *CAVS* categories is “CityAlley”, featuring a slow upward pan in an alley. Its bitrate is reduced from 1,260.1 to 128.3 kbps by CAVS (90% savings). The VMAF scores of CAVS for this video are 56.8, 69.6, and 73.2, with tile counts of 4.9, 25.9, and 33.4 across the 3 GPU capacities. Overall, CAVS demonstrates consistent performance with varying bitrates and content without significant quality degradation.

VI. CONCLUSION

We have developed a computation-adaptive video streaming system, namely CAVS, to address the storage and reliability issues in bandwidth-adaptive streaming. CAVS transmits low-resolution video and partially recovers it at the client side based on the available computational resources and the saliency weights of the video tiles. Our evaluation results demonstrate savings in bandwidth consumption and the ability to adapt to varying computation capacities to maximize the quality of experience and maintain the real-time streaming performance. The project repository of CAVS is at <https://github.com/hwsel/CAVS>.

ACKNOWLEDGMENT

This work was supported in part by the National Science Foundation under award 2350188.

REFERENCES

- [1] T.-Y. Huang, R. Johari, N. McKeown, M. Trunnell, and M. Watson, “A buffer-based approach to rate adaptation: Evidence from a large video streaming service,” in *SIGCOMM*, 2014, pp. 187–198.
- [2] X. Yin, A. Jindal, V. Sekar, and B. Sinopoli, “A control-theoretic approach for dynamic adaptive video streaming over HTTP,” in *SIGCOMM*, 2015, pp. 325–338.
- [3] H. Mao, R. Netravali, and M. Alizadeh, “Neural adaptive video streaming with Pensieve,” in *SIGCOMM*, 2017, pp. 197–210.
- [4] “Information technology - dynamic adaptive streaming over HTTP (DASH),” 2022, ISO/IEC 23009-1:2022.
- [5] “HTTP live streaming,” 2017, RFC 8216.
- [6] A. Bentalb, B. Taani, A. C. Begen, C. Timmerer, and R. Zimmermann, “A survey on bitrate adaptation schemes for streaming media over HTTP,” *IEEE Communications Surveys & Tutorials*, vol. 21, no. 1, pp. 562–585, 2019.
- [7] C. Dong, C. C. Loy, K. He, and X. Tang, “Image super-resolution using deep convolutional networks,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 2, pp. 295–307, 2016.
- [8] W. Shi, J. Caballero, F. Huszar, J. Totz, A. P. Aitken, R. Bishop, D. Rueckert, and Z. Wang, “Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network,” in *CVPR*, 2016, pp. 1874–1883.
- [9] R. Wu, W. Bao, L. Ge, and B. B. Zhou, “ASRSR: Adaptive sending resolution and super-resolution for real-time video streaming,” in *Q2SWinet*, 2023, pp. 61–68.
- [10] N. Li, Z. Zhu, S. Wei, and Y. Liu, “EVASR: Edge-based saliency-aware super-resolution for enhanced video quality and power efficiency,” *ACM Transactions on Multimedia Computing, Communications, and Applications*, pp. 1551–6857, 2025.
- [11] K. Ogata, “PID controllers and modified PID controllers,” in *Modern Control Engineering*, 2010.
- [12] “Flask,” 2024. [Online]. Available: <https://github.com/pallets/flask>
- [13] “FFmpeg,” 2023. [Online]. Available: <https://www.ffmpeg.org/>
- [14] C. Chen, G. Wang, C. Peng, Y. Fang, D. Zhang, and H. Qin, “Exploring rich and efficient spatial temporal interactions for real-time video salient object detection,” *IEEE Transactions on Image Processing*, vol. 30, pp. 3995–4007, 2021.
- [15] “H.265: High efficiency video coding,” 2021. [Online]. Available: <https://www.itu.int/rec/T-REC-H.265>
- [16] “STVS,” 2023, <https://github.com/guotaowang/STVS>.
- [17] M. Everingham, S. A. Eslami, L. Van Gool, C. K. Williams, J. Winn, and A. Zisserman, “The PASCAL visual object classes challenge: A retrospective,” *International Journal of Computer Vision*, vol. 111, no. 1, pp. 98–136, 2015.
- [18] A. Mercat, M. Viitanen, and J. Vanne, “UVG dataset: 50/120fps 4K sequences for video codec analysis and development,” in *MMSys*, 2020, pp. 297–302.
- [19] “NVIDIA system management interface,” 2024. [Online]. Available: <https://developer.nvidia.com/system-management-interface>
- [20] “VMAF,” 2025. [Online]. Available: <https://github.com/Netflix/vmaf>